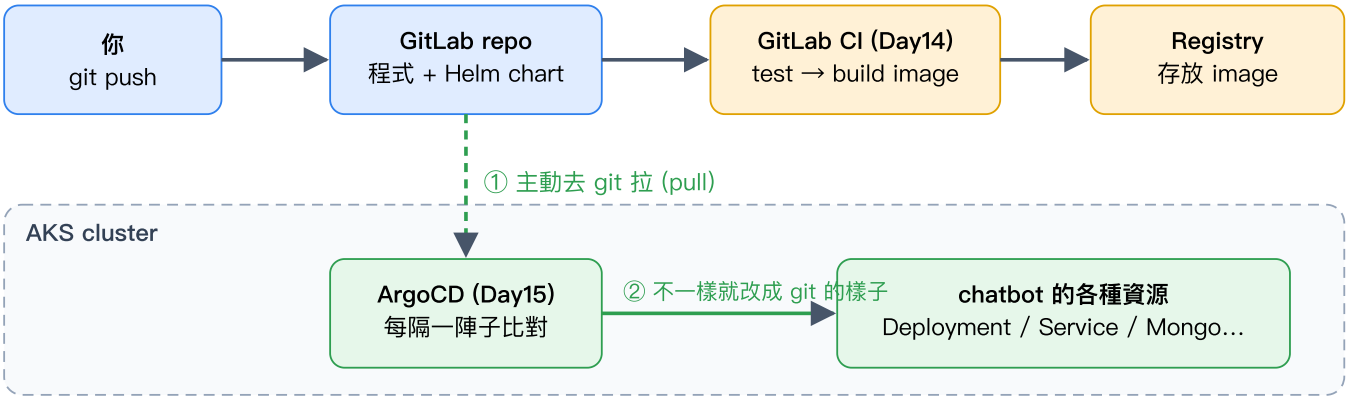
Day 14–15:CI 與 GitOps(ArgoCD)圖解

今天做了什麼 · 面試怎麼說 · 面試會考什麼

一句話總結

Day 14 做了 **CI**:你每次 push 程式,GitLab 會自動跑測試、建 image;測試紅了就不會建。
Day 15 做了 **CD 的前半**:在 cluster 裡裝 **ArgoCD**,讓它盯著 git,**cluster 自動長成 git 裡寫的樣子**。

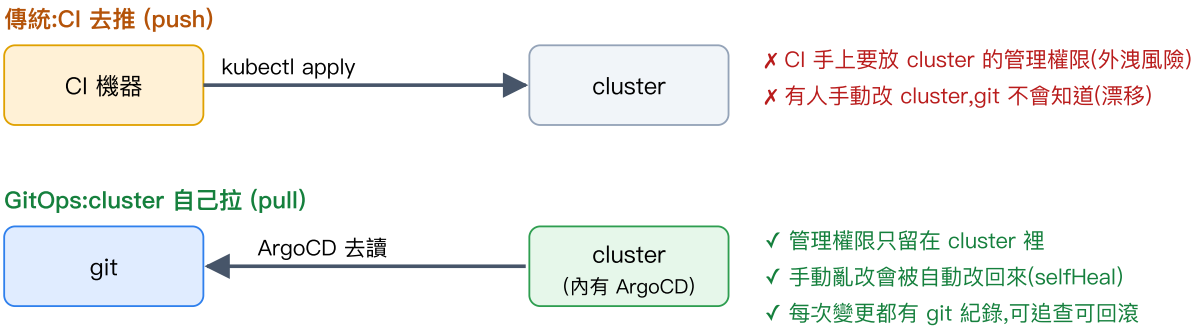
1. 整條路現在長什麼樣



目前的銜接缺口(Day 16 要補):CI 把 image 放在 GitLab registry,但 cluster 現在跑的還是 ACR 的舊 image。Day 16 會改 Helm 的 image 設定、讓 cluster 能拉 GitLab registry,把這條路真正接起來。

2. ArgoCD 是什麼、為什麼要用

它是一支**跑在 cluster 裡的程式**,工作只有一個:不斷比對「git 裡寫的」和「cluster 現在的」。不一樣就把 cluster 改成 git 的樣子。這個做法叫 **GitOps:git 是唯一的真相**。



3. 今天的操作步驟(照順序)

#	做了什麼	為什麼
1	啟動 AKS(`az aks start`)	cluster 平常關著省錢,要用才開
2	用 Helm 安裝 ArgoCD 到 `argocd` 命名空間,設定寫在 `argocd/values.yaml`	單一小 node 資源有限,關掉 SSO / 通知 / ApplicationSet,把資源 request 壓低
3	你建了一個 GitLab token,用 `kubectl create secret` 存進 cluster,並貼上標籤 `argocd.argoproj.io/secret-type=repository`	你的 repo 是**私有**的,ArgoCD 需要唯讀憑證才讀得到;標籤是 ArgoCD 認得「這是 repo 憑證」的暗號
4	`kubectl apply -f argocd/application.yaml`(**只做這一次**)	告訴 ArgoCD「請讓 cluster 長得跟這個 repo 的 mychart 一樣」。這是起手式(bootstrap),之後都走 git
5	檢查:Application 顯示 **Synced / Healthy**,15 個資源全綠,Pod 沒被重啟	證明 ArgoCD 渲染出的結果跟你原本 Helm 部署的一模一樣,只是「接管」

token 為什麼我沒經手:token 是密碼等級的東西,所以由你自己在終端機貼上,不透過對話。這也是面試可以提的小細節(憑證不進 git、不進對話)。

4. Application 檔案逐行看

```
spec:
  source:
    repoURL: https://gitlab.com/...            # 去哪個 repo 讀
    targetRevision: main                       # 讀哪個分支
    path: mychart                              # repo 裡 Helm chart 在哪
  destination:
    server: https://kubernetes.default.svc     # 部署到「自己所在的 cluster」
    namespace: default
  syncPolicy:
    automated:
      prune: true      # git 刪了,cluster 也刪
      selfHeal: true   # 有人手動改,自動改回 git 的樣子
```

ArgoCD 的狀態怎麼讀

Synced	OutOfSync	Healthy	Degraded
cluster 跟 git 一致	git 有新內容或被手改	Pod 真的跑起來了	部署了,但跑不起來

⚠ **Synced 不等於 Healthy**:Synced 只說「我照 git 套用了」;image 拉不到、程式一啟動就掛,都會是「Synced 但 Degraded」。

兩個維度要分開看:一個問「跟 git 一樣嗎」,一個問「真的活著嗎」。

5. 面試可以這樣說

30 秒版本

「我用 GitLab CI 做測試和建 image,測試失敗就不會建出 image。部署端用 ArgoCD 做 GitOps:cluster 裡的 ArgoCD 會主動去 git 拉 Helm chart,發現跟 cluster 不一樣就自動同步。這樣部署憑證不用放在 CI 上,而且 cluster 的狀態都能在 git 歷史裡追查和回滾。」

2 分鐘版本(加上取捨與細節)

「Application 開了 `selfHeal`,有人手動 kubectl 改線上設定會被改回 git 的樣子,所以 git 是唯一真相。開了 `prune`,git 刪掉的資源 cluster 也會刪,不會留孤兒。

我的 repo 是私有的,所以用唯讀 deploy token 當 repo 憑證,存成 cluster 裡的 Secret,沒有進 git。

接管時我先確認 ArgoCD 渲染出來的結果跟原本 Helm 部署一致,所以接管的當下 Pod 沒有重啟、Mongo 資料沒動,是安全的切換。」

被追問「這個規模有必要用 ArgoCD 嗎?」要誠實:

「單人單 cluster 其實用 CI 直接 `helm upgrade` 也能做。ArgoCD 的價值是:憑證不離開 cluster、能偵測並修正漂移、部署狀態有 UI 可看、多環境時好擴充。我選它是為了練習 GitOps 這個做法,不是因為規模需要。」

6. 面試會考什麼(含建議答案)

Q1. GitOps 是什麼?跟傳統 CI/CD 差在哪?

> 以 git 為唯一真相,由 cluster 內的 agent「拉」設定並持續比對。傳統是 CI 機器「推」(kubectl apply),CI 要持有 cluster 權限,而且手動改線上不會被發現。

Q2. push 模型和 pull 模型各有什麼優缺點?

> push 簡單直觀,但 CI 要有 cluster 憑證、無法偵測漂移。pull 憑證留在 cluster 內、能持續校正,代價是多裝一個元件要維護,出問題多一層要排查。

Q3. selfHeal 和 prune 是什麼?風險是什麼?

> selfHeal:被手動改動時自動改回。prune:git 刪掉的資源 cluster 也刪。風險是 prune 開著時,誤刪 git 裡的檔案會連線上資源一起刪,所以要靠 code review / 分支保護把關;緊急手動修正也會被改回,要先改 git。

Q4. Synced 和 Healthy 有什麼不同?

> Synced = cluster 和 git 一致。Healthy = 資源真的在正常運作。可以「Synced 但 Degraded」,例如 image 拉不到。

Q5. 為什麼 ArgoCD 的憑證要用 Secret?token 洩漏怎麼辦?

> 憑證不進 git、不進對話。我給的是唯讀權限(最小權限),最壞情況是被讀 repo,不能改 code。洩漏就到 GitLab 撤銷該 token、重建、更新 Secret。

Q6. 你怎麼確保接管現有部署時不會出事?

> 先確認原本 Helm 沒有額外 `--set`,ArgoCD 渲染結果等於現況;接管後檢查 Pod 存活時間沒重置、狀態 Synced/Healthy。

Q7. 誰來更新 image tag?CI 還是人?

> (Day 16 會實作)CI 建好 image 後,把新 tag 寫回 git 裡的 values,ArgoCD 偵測到 git 變了就部署。部署動作全由 git 變更觸發,沒人下 kubectl。

7. 已知限制(誠實列出,面試加分)

限制	說明 / 之後怎麼辦
舊 Helm release 紀錄還在	cluster 裡還留著 `helm list` 的 chatbot 紀錄,但之後不該再用 `helm upgrade`,否則會跟 ArgoCD 打架。文件裡要註明。
ArgoCD UI 只用 port-forward	沒有對外開放、沒有 TLS(`server.insecure` 只適合練習)。正式環境要放 Ingress + TLS + SSO。
內建 admin 帳號與初始密碼	練習方便,正式環境要關掉 admin、改 SSO,並刪除初始密碼 Secret。
repo 憑證是個人 token	token 綁個人帳號。正式做法用專案層級的 deploy token,並定期輪替。
image 來源尚未統一	CI 推 GitLab registry,cluster 目前仍用 ACR 舊 image。Day 16 處理(imagePullSecret)。
CI 到部署還沒全自動	目前 CI 建完 image 後,更新 git 裡 tag 的步驟還沒做,Day 16 完成後才是真正的端到端。

8. 一頁複習卡

名詞	一句話
CI	每次 push 自動測試、建 image(Day 14)
GitOps	git 是唯一真相,cluster 自動跟上
ArgoCD	cluster 內的程式,持續比對 git 與 cluster
Application	ArgoCD 的設定單位:從哪讀、部署去哪、怎麼同步
Synced / Healthy	跟 git 一致 / 真的活著(兩件事)
selfHeal / prune	手改自動改回 / git 刪了 cluster 也刪
bootstrap	唯一要手動 apply 的那一次,之後全走 git
漂移 (drift)	cluster 實際狀態偏離 git 的情況